air-classifier-service — ARCHITECTURE & DESIGN

IN THIS REPO · +PROD PRODUCTION REQUIREMENT · EXT OTHER PROJECT / MANAGED

TEXT CLASSIFICATION FOR THE AIR PLATFORM · FOUR-TIER ESCALATION LADDER · REV C · AS BUILT · 2026-11-13 · SUPERSEDES REV B

PLATFORM NETWORK — EAST-WEST ONLY · CALLERS AUTHENTICATE WITH X-API-KEY · /metrics REACHABLE BY PROMETHEUS ONLY

AIR-PLATFORM

chat entry point for external applications · calls /v1/classify per message

auth: X-API-Key

AIR-CLIENT

typed client, generated from this repo's **docs/openapi.json** — the API contract

PEER SERVICES

review ingest · feedback pipelines · CRM & support tooling · use /classify/batch and /summary/refresh

ONE KEY, THREE CONTROLS

Each API key carries **scopes**, a **max_tier** ceiling (e.g. keep a caller on-prem at T2) and its own **rate limit**. Keys are stored only as salted SHA-256 hashes.

air-classifier-service — OWN REPO · OWN IMAGE · STATELESS REPLICAS

Sentiment · tone · topics · intent · urgency · routing · requires-human · actionability · aspects · rating consistency — whichever the text and the ladder's reach support. Every response records how far it climbed and why (**escalation_trace**).

API — /V1 (pinned by contract test)

```
POST /v1/classify          → one verdict
POST /v1/classify/batch    → ≤ 100 verdicts, sync
POST /v1/summary/refresh   → customer summary
GET  /v1/health · /v1/ready · /v1/capabilities
GET  /metrics
```

errors: RFC 9457 problem+json · only 503 = no tier answered

ADMISSION api/ · security/

- X-API-Key → key record: scopes, max_tier, quota
- per-key token-bucket rate limit → 429 + Retry-After
- Idempotency-Key replay · 409 on a changed body
- body-size cap · X-Request-ID echoed & forwarded

ANALYSISSERVICE pipeline/service.py — the only place text is prepared and results are derived

normalise ▶ redact PII ▶ detect language ▶ cache lookup — hit returns here ▶ Orchestrator climbs the ladder ▶ derive urgency · routing · requires_human · actionability · rating_consistency

Tiers only ever see redacted text. Derived fields are deterministic rules on the winning verdict, never a model output.

ESCALATION LADDER pipeline/orchestrator.py + policy.py + tiers/ — cheapest first, climb only when not confident

T0 · rules	T1 · classifier	T2 · local LLM	T3 · cloud LLM
lexicons · regex · emoji · explicit rating	zero-shot NLI (ONNX) + lexicon ensemble	gateway class **local** · schema-constrained · adds tone/topics/intent	gateway class **auto**; below 0.65 confidence, arbiter class **cloud**
< 1 ms · free	80–200 ms · free	0.2–2 s · own hardware	0.8–3 s · billed

EscalationPolicy, checked in this fixed order at every rung:

1 tier ceiling · 2 latency budget · 3 cost budget · 4 confidence floor · 5 margin floor · 6 complexity triggers · 7 tier disagreement · 8 unsupported language · 9 next tier available?

RESILIENCE

Per LLM rung: timeout · circuit breaker · bulkhead. A failed rung becomes a failed trace step, never an exception.

SUMMARY REFRESH

Classifies new items via the ladder, merges a deterministic rollup, rewrites the narrative with one guarded gateway call. Stores nothing.

OBSERVABILITY

structlog JSON · Prometheus **air_classifier_*** · optional OpenTelemetry spans per tier and gateway call.

AIR-MODEL-GATEWAY EXT

every model call · **POST /v1/generate** · :8083

- this service asks for a **class:** local · auto · cloud — never a model or vendor
- gateway owns provider keys, routing & failover, cost, caching
- daily budget for this service (429 budget_exceeded)

REDIS EXT VIA AIR-INFRA

shared verdict cache across replicas · optional — falls back to an in-process cache

PROMETHEUS / GRAFANA EXT

scrapes /metrics · owned by air-infra

MONGODB EXT OPTIONAL

audit log of jobs (redacted text + verdict, TTL) · off by default

TIER-1 ONNX ARTIFACT

exported by scripts/export_zeroshot_model.py · mounted at /opt/models or baked in · absent → lexicon fallback

ALL OPTIONAL

A bare checkout with nothing configured still boots and serves on T0 + T1. A missing dependency degrades the answer, it never fails it.

DESIGN PRINCIPLES — WHY IT IS BUILT THIS WAY

01 · PAY ONLY FOR THE DIFFICULTY YOU HAVE

Most real text ("Great product, thanks!") is trivially classifiable; a cloud LLM costs orders of magnitude more than it needs to. Every request enters at T0 and climbs only when the rung below isn't confident — so the bill follows the hard cases, not the volume.

02 · DEGRADE, NEVER ERROR

A timeout, breaker trip or gateway outage turns that rung into a failed trace step; the best lower-tier verdict is returned with **degraded: true**. The only 503 is "no tier produced anything". Readiness stays 200 while T0/T1 can answer.

03 · EVERY DECISION IS EXPLAINABLE

escalation_trace lists each rung's label, confidence, latency and the reason it climbed. EscalationReason is a closed set — it is also a Prometheus label, so dashboards show why traffic escalates.

04 · THRESHOLDS ARE CONFIG, NOT CODE

Confidence floors, margins, latency and cost budgets are all **AIR_CLASSIFIER__*** settings. Retuning the cost/accuracy trade never needs a logic deploy. Per request: min_tier · max_tier · max_cost_usd (default $0.05).

05 · ONE MODEL GATEWAY, NO VENDOR CODE

No provider SDKs or keys in this repo. Vendor choice, failover and spend caps live in air-model-gateway's routing config, per class, with an override for this service. Swapping a model is a gateway change, not a release here.

06 · PRIVACY BEFORE ANYTHING PERSISTS

PII is redacted before the cache key, before any tier and before any log line. Raw text is never logged in production (startup refuses it). The service is stateless; summaries are stored by the caller.

MODEL ACCESS — CLASS PER CALLER

CALLER	CLASS	ROUTING FOR THIS SERVICE (GATEWAY OVERRIDE)
Tier 2	**local**	single local rung, no fallback — an outage fails fast and this ladder escalates
Tier 3	**auto**	cheap cloud models with cross-vendor failover
T3 arbiter	**cloud**	strongest models, only when T3 confidence < 0.65
Summary	**local**	failure keeps the previous narrative, degraded: true

Concrete models per class: gateway **GET /v1/capabilities**. Request body is closed (extra="forbid"): an unknown field is a 422 on every call.

GATEWAY RESPONSE → LADDER BEHAVIOUR

GATEWAY SAYS	LADDER DOES
5xx · timeout · unreachable	capacity gap → climb on, or degrade
429 rate_limited	rung fails, ladder continues
429 budget_exceeded	rung fails until 00:00 UTC (daily budget spent)
other 4xx	rung recorded **failed**, error log — a config/contract bug here
refusal · bad JSON	one repair retry for schema errors, then fail

The gateway retries its own chain once; this service does not retry on top.

RESPONSE CONTRACT

ALWAYS PRESENT

```
sentiment         label · polarity · confidence
urgency           derived
routing           queue · priority (derived)
requires_human    derived
actionability     derived
decided_by        t0_rules … t3_cloud_llm
degraded          true if it wanted to climb
escalation_trace  one step per rung tried
usage             tiers · tokens · est_cost_usd
```

ONLY WHEN THE LADDER REACHES T2/T3

```
tone     label · confidence
topics   [label · confidence]
intent   label · confidence
```

Need real language understanding, so T0/T1 never guess them. Callers that always need them send options.min_tier = t2_local_llm.

WHEN THE CALLER SUPPLIES CONTEXT

```
rating_consistency   with rating
aspects              per-aspect polarity
```

VOCABULARY & COMPATIBILITY

- All labels are enums in **constants.py:** Label, Tone, Topic, Intent, Urgency, RoutingPriority, Actionability, RatingAgreement
- Adding a member is safe; removing or re-meaning one is breaking and bumps **CACHE_SCHEMA_VERSION**
- **docs/openapi.json** is committed; air-client generates from it; a contract test fails on drift
- External names kept on purpose: env prefix **AIR_CLASSIFIER__**, metric prefix **air_classifier_**

DEPLOYMENT & OPERATIONS

IMAGE

- python:3.12-slim · multi-stage · non-root user **air** (uid 1001)
- container port **8080** (local host port 8082)
- one uvicorn worker per container — scale with replicas
- built-in HEALTHCHECK on /v1/health
- 12-factor: every setting is an **AIR_CLASSIFIER__…** env var

PROBES & RUN MODES

- **liveness** → /v1/health (process only)
- **readiness** → /v1/ready: ok / degraded = 200; 503 only when no tier can answer — never pull a pod for a degraded state
- **make run** — native dev loop on :8082
- **make up** — container on air-infra's air-net + Redis
- **make up INFRA=none** — standalone, in-process cache
- **make doctor** — checks every dependency

PRODUCTION CHECKLIST +PROD

- APP__ENV=production (refuses unauthenticated mode and raw-text logging)
- real SECURITY__HASH_SALT + key hashes — the dev salt is not auto-rejected
- MODEL_GATEWAY__TOKEN + base URL; policy & routing entries in the gateway repo
- CACHE__BACKEND=redis when running > 1 replica
- Tier-1 ONNX artifact mounted or baked in
- /metrics restricted to Prometheus · JSON logs · OTel on

WHAT CHANGED SINCE REV B

AREA	REV B (PLAN)	REV C (AS BUILT)
Model access	~~earlier model service client, batch pool only~~	air-model-gateway /v1/generate, by model class, for T2/T3 and summaries
Classification core	~~trained head per attribute, calibrate, map~~	four-tier escalation ladder with an explainable trace
Label contract	~~/v1/taxonomy endpoint, spec/taxonomy~~	enums in constants.py, published via docs/openapi.json
Batch	~~async jobs, polling, object store~~	synchronous /v1/classify/batch, ≤ 100 items
State	~~Postgres: jobs, keys, usage, feedback~~	stateless; keys from env/file; optional Mongo audit
Capacity	~~interactive + batch pools, priority~~	one pool; per-key rate limit and max_tier
Source of truth	~~spec/ generates code~~	code-first; OpenAPI generated and contract-tested
New	—	/v1/summary/refresh; /v1/capabilities; health/ready under /v1
Name	~~air-classifier~~	air-classifier-service

OPEN ITEMS FOR THE TEAM

- No CI workflow or Kubernetes manifests in the repo yet — **make check** is the intended CI gate
- 5 golden-set T1 cases fail on the lexicon fallback; re-check with the ONNX artifact
- 5 older files need one **make fmt** commit
- No automated check that the gateway's routing override for this service is still in place
- Production guard should also reject the dev hash salt
- X-Remaining-Budget-Ms is sent but not yet read by the gateway

Details: README.md · docs/01-hld.md · docs/02-lld.md (§12 changelog) · CLAUDE.md